Kafka: Actually Running It + Advanced Topics (Days 48, 58-61)

The producer/consumer code already exists and compiles. It has **never been run against a live broker**. Do this first before anything else in this doc — it proves everything already written actually works.

Part A — Run it for real (closes out Day 48)

```
cd notification-service
docker compose up -d          # starts Kafka using the docker-compose.yml already there
docker ps                     # confirm the "notification-kafka" container is Up
```

Start both apps (banking app on 8080, notification-service on 8081 — set `DB_PASSWORD`, `JWT_SECRET` for the banking app; `DB_PASSWORD` for notification-service too).

End-to-end test: 1. Register + login on the banking app, create a customer + two accounts. 2. `POST /api/v1/transactions/transfer` between them. 3. Watch notification-service's console — you should see Kafka consumer log lines picking up the `transaction.completed` event. 4. `GET /api/v1/notifications?userId=<the customer's user id>` on notification-service (port 8081) — confirm a `Notification` row exists with `notificationStatus: SENT` (or `FAILED` if SMTP creds still aren't set — that's expected, see the deferred-email memory). 5. Same test with `PATCH /api/v1/beneficiaries/{id}/activate` on the banking app — confirm `beneficiary.activated` arrives too.

If step 3 shows nothing: check `docker ps` again (container may have exited), check both apps' `spring.kafka.bootstrap-servers` actually say `localhost:9092`, and check for `transaction. completed` vs `transaction.completed` typos (a stray space creates a silently different topic — this exact bug happened once already).

Part B — Dead-letter queue / retry topic (Day 58)

When a consumer throws repeatedly, don't let it block the partition forever. Add a `DefaultErrorHandler` with a dead-letter recoverer in `notification-service`:

```
@Configuration
public class KafkaConsumerConfig {

    @Bean
    public DefaultErrorHandler errorHandler(KafkaTemplate<String, String> template) {
        var recoverer = new DeadLetterPublishingRecoverer(template,
                (record, ex) -> new TopicPartition(record.topic() + ".DLT", record.partition()));
        return new DefaultErrorHandler(recoverer, new FixedBackOff(2000L, 3));
        // retries 3 times, 2s apart, then publishes to "<topic>.DLT"
    }
}
```

Test: temporarily make `TransactionEventConsumer` throw on purpose (`throw new RuntimeException("test")`), publish an event, confirm it retries 3 times in the logs, then confirm a message

lands on `transaction.completed.DLT` (listen to it manually with a throwaway `@KafkaListener(topics = "transaction.completed.DLT")` or the `kafka-console-consumer` CLI inside the container: `docker exec -it notification-kafka /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic transaction.completed.DLT --from-beginning`).

Part C — Consumer groups & scaling (Day 59)

Both consumers already share `group-id=notification-service`. To prove group behavior: run **two instances** of notification-service on different ports (`--server.port=8082`), same group id. Publish several events — you should see them split between the two instances' logs, not duplicated. This is the core guarantee: one message, one consumer *within* a group.

Part D — Monitoring consumer lag (Day 60)

```
docker exec -it notification-kafka /opt/kafka/bin/kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 --describe --group notification-service
```

Shows `LAG` per partition — how many messages are produced but not yet consumed. Stop notification-service, publish a few transfers from the banking app, run this command again — LAG should rise. Restart notification-service, run it again — LAG should drop back to 0 as it catches up.

Part E — Async notification review (Day 61)

This is just "re-run Part A's end-to-end test one more time, calmly, and write down what you observed" — confirms the whole pipeline (produce → broker → consume → notification record → email attempt) in one sitting once everything above is understood.